JOULE · STRATEGIC PLANNING DOCUMENT

From Validated Engine to Smart-Gym Product

A product specification and technical roadmap for turning Joule's camera-based exercise-tracking engine into "Joule Coach" — a subscription smart-gym product where the gym's own cameras act as each member's personal trainer.



Prepared	**6 August 2026**
Scope	**Part 1 — Product Specification | Part 2 — Technical Roadmap (Phase F–L)**
Status	**Planning document — nothing described below has been built**
Source repo	**joule (PRODUCT_SPEC.md & PRODUCT_ROADMAP.md)**

Grounded throughout in the current, real state of the Joule codebase and its existing validation record (RepCount, CMU Panoptic, Fit3D) as documented in `ROADMAP.md` and `STATUS.md`. Every technical claim in Part 2 traces to a specific file in the repository.

Contents

1. Executive Summary — purpose, the one-paragraph version, sequencing at a glance, four decisions the team needs to make
2. Part 1 — Product Specification: Joule Coach
    a. User personas and jobs-to-be-done
    b. Feature set (MVP / v1 / v2+)
    c. Design direction
    d. Sprint plan (pipeline-validated → multi-site product)
    e. Evaluation criteria
    f. Honest scope boundary
3. Part 2 — Technical Productization Roadmap (Phase F Onward)
    a. Verified current state of the codebase
    b. Gap analysis
    c. Phased engineering roadmap (Phases F–L)
    d. Architecture recommendations
    e. Key technical risks

Executive Summary

Purpose

This document lays out how Joule goes from a **validated computer-vision engine** to a **commercial smart-gym product** — a subscription service where the gym's own cameras act as each member's personal trainer: automatic exercise tracking, right/wrong form feedback with an accuracy score, an in-app plan builder with real adherence tracking (not self-report), and a library of demo videos.

It is built from two companion planning passes, included in full below:

- **Part 1 — Product Specification.** Who this is for, what ships at MVP vs. later, what the app should look and feel like, the sprint sequence from "engine validated" to "multi-site product," and how we'll know it's actually working.
- **Part 2 — Technical Roadmap.** What has to be built underneath the app — persistence, identity, privacy/compliance, a live floor runtime, backend, and multi-tenant scale-out — sequenced as Phases F through L, continuing this project's existing Phase A–E numbering. Every claim in this part is traced to a specific file in the current codebase, not generic advice.

The one-paragraph version

Joule's underlying pipeline (person tracking, pose estimation, rep counting, form scoring, 3D fusion) is validated against public benchmarks and about to be validated on real gym footage. Turning it into "Joule Coach" — a paid product — is not a thin app wrapped around the existing CLI. Four things decide whether this becomes a real business: **(1)** whether accuracy holds up on a real, crowded, mirror-walled gym floor, **(2)** whether it runs fast enough on affordable edge hardware, **(3)** whether member identity can be reliably held for a full visit without becoming a privacy liability, and **(4)** getting consent and biometric- data compliance right — which today's engine is well-positioned for (nothing is currently persisted) but which every step toward a real product puts at risk. The roadmap below sequences engineering so those four risks get answered with a single, low-cost pilot gym before any larger commitment — and the product spec keeps every member-facing promise honest about what the engine can and can't yet do.

How to use this document

Part 1 (Product Specification) is written for anyone thinking about the member experience, the business model, or go-to-market. Part 2 (Technical Roadmap) is written for engineering leadership and anyone evaluating build cost, timeline, or technical risk. Read Part 1 first for the "what and why," then Part 2 for the "how and what it takes."

The core sequencing, at a glance

```
Phase D  Real-footage validation (engine team, in progress)
Phase E  RTSP sync / Jetson port / kiosk check-in
              |
Phase F  Persistence & identity spine        -\
Phase G  Privacy & compliance gate             |- parallel, no gym required
Phase L  Model & coverage expansion (ongoing) -/
              |
Phase H  Always-on floor runtime (absorbs E1/E2)
              |
Phase I  Backend API + operator console + FIRST PILOT GYM (absorbs E3)
              |
Phase J  Member app: plans, adherence, video library
              |
Phase K  Multi-tenant scale-out, billing, fleet ops
```

Product Sprints 0–6 (Part 1, Section 4) map onto Phases F–K: Sprint 0 runs alongside Phase D/F; Sprints 1–3 land inside Phase I at the pilot gym; Sprint 4 is Phase I's exit; Sprints 5–6 are Phases J/K.

The four decisions this document asks the team to make

1. **Commit to "edge inference, not cloud"** for the floor runtime — for compliance, bandwidth, and failure-mode reasons detailed in Part 2, Section 3.3. This is close to irreversible once built around.
2. **Commit to "bind identity at check-in, never persist appearance embeddings across visits"** — the current engine already does this by default; the temptation will be to "just save the gallery" for convenience. Don't. See Part 2, Section 3.4.
3. **Fund Phase G (privacy/consent/compliance) as a real, gated phase**, not a legal-review afterthought before launch. It cannot be retrofitted once member data has been collected without proper consent.
4. **Hold the line on the MVP's honest scope boundary** (Part 1, Section 6): no sit-up counting, no head-on bench press counting, no mirror coverage, no absolute "correctness %" until it's actually validated. Every one of these will face pressure to be quietly promised early — don't let marketing outrun what Phase D and Phase I5 actually prove.

PART 1

Product Specification

Joule Coach: personas, MVP-to-v2 feature set, design direction, sprint plan, and evaluation criteria for a subscription smart-gym product.

Product Specification: Joule Coach — Camera-Native Personal Training for Smart Gyms

Written 2026-08-06. Expanded from the founder's one-line brief: "Smart gyms where members don't need a human personal trainer — they pay a subscription, and the gym's cameras + AI act as their personal trainer via a companion app: real-time form/accuracy feedback per exercise, an in-app workout plan builder with automatic adherence tracking, and a library of sample/demo videos teaching correct exercise form."

This is a standalone strategic planning document for the human product/ engineering team. It is grounded in the current, real state of the underlying Joule computer-vision engine (per `ROADMAP.md` / `STATUS.md` as of 2026-08-06): component-level validation against RepCount, CMU Panoptic, and Fit3D is done and credible; Phase D (real gym footage validation) is the next engine milestone and has not yet run. The product plan below is sequenced so the app can start being built in parallel with Phase D/E, but its scope claims never outrun what the engine can actually promise on day one. It is a companion document to `PRODUCT_ROADMAP.md` (the technical/ systems architecture side of the same plan).

---

1. Personas and Jobs-to-be-Done

1.1 Gym Member (primary persona) — three practical sub-segments

The "member" persona is not monolithic; the product needs to serve three distinct relationships to the product honestly, especially because the engine's coverage is exercise-specific and camera-layout-dependent at launch.

A. The New/Anxious Member ("Never had a trainer, doesn't know if I'm doing this right") - JTBD: "Tell me, in the moment or right after, whether I'm about to hurt myself or waste my time, without the social cost of asking a stranger." - Trust is the whole game for this segment — they will abandon at the first moment the app is confidently wrong.

B. The Committed Self-Directed Member ("I know the movements, I want structure and proof I'm sticking to it") - JTBD: "Build me a plan, hold me accountable to it with real data (not my own guesswork/self-report), and show me I'm progressing." - This segment is the primary buyer of the adherence-tracking and analytics value proposition.

C. The Data-Driven Advanced Member ("I already train seriously, I want the numbers") - JTBD: "Give me joint-angle-level detail, asymmetry and fatigue trends over months, and let me export/compare — don't dumb this down." - Lowest onboarding friction, highest expectation of accuracy and depth; also the segment most likely to notice and forgive known model limitations if the app is transparent about them.

1.2 Gym Owner/Operator (primary business persona)

- JTBD: "Differentiate my gym and justify a premium tier or higher price point, without hiring more PTs, while proving the system pays for itself in retention within a budget cycle."
- Secondary JTBD: "Know immediately if a camera goes down or a station is miscalibrated, before it silently degrades every member's experience for weeks."

1.3 Front-of-House / Floor Staff (supporting persona)

- JTBD: "When the app is wrong, confused, or a member is frustrated at the kiosk, give me a two-minute fix, not a support ticket queue."
- This role exists because a 100%-automated system with zero human backstop is where trust erodes fastest — floor staff are the manual override, not a redundant PT.

1.4 Remote Coach/Reviewer (premium, human-in-the-loop persona —

deliberately added) - JTBD: "Let me spend my time on the 5% of cases that need a human — ambiguous form calls, injury-risk escalations, and plan customization for premium members — instead of watching every rep of every member." - Why this role matters strategically: the brief promises members won't need "a human personal trainer," but the engine has real, named blind spots (sit-ups uncounted, bench press needs a side angle, mirrors are a blind spot, out-of-frame limbs break tracking). A lightweight human-review layer for premium tiers and for injury-risk escalations is what makes the "no human trainer needed" claim honest rather than reckless — it converts unhandled edge cases into a monetizable premium feature instead of a silent failure mode.

---

2. Feature Set (by persona, by priority)

Organizing principle: **MVP ships only what the engine has already proven** (standing/unencumbered movements validated cleanly on Fit3D: squat, deadlift, curls, rows, presses — squat carries a known larger error band and should ship with a visible confidence caveat, not silence). Bench press and sit-ups are explicitly *not* MVP claims. Mirrors are excluded by installation rule, not by a feature.

2.1 MVP (Sprint 1–2 scope — "first pilot gym")

#	Feature	Persona	Description & Acceptance Criteria
1	**QR/kiosk camera check-in**	Member, Staff	Member scans a personal QR code (or taps a phone-free kiosk badge) at a fixed check-in camera; system binds their name to a fresh Re-ID embedding for that visit (leverages the existing `/checkin` binding pattern). Acceptance: check-in confirmation appears within 3s; a failed match routes to a staff-assisted manual bind, never a silent misattribution.
2	**Guided session start ("what am I about to do")**	Member	Before a set, member selects the exercise from a *closed list* of currently-supported movements in-app (squat, deadlift, bicep curl, row, overhead press, lateral raise) — no free-text/open-world exercise entry in MVP, because the classifier's trained coverage is finite. Acceptance: unsupported exercises (e.g., sit-ups, bench press) are visibly labeled "Coming soon" in the picker, not hidden — sets expectations rather than producing a silent bad read.
3	**Post-set form report (near-real-time, not mid-rep live overlay)**	Member	Within 2–5 seconds of a set ending, member sees: rep count, per-rep depth/ROM validity, and a plain-language form score vs. the reference "good execution" signature, with the 1–2 biggest deviation cues (e.g., "knees tracking inward on reps 4–6"). Acceptance: every report explicitly states whether 3D (fused, higher-confidence) or 2D single-camera (lower-confidence) was used, never presented as equally certain.
4	**"Stay in frame" real-time cue**	Member	A simple, low-latency (not form-accuracy-dependent) in-gym signal — light/tone on an equipment-mounted indicator or a phone push — when the member's feet or body leave the calibrated camera volume mid-set, since full-body-including-feet-in-frame is a hard tracking requirement. Acceptance: this cue fires on frame-exit within one video frame's latency budget, independent of and faster than the form-scoring pipeline.
5	**Manual workout plan builder (template + custom)**	Member	Member assembles a weekly plan from a fixed library of supported exercises (sets/reps/target tempo). Acceptance: plan builder blocks adding unsupported exercises with an explanation, and flags any planned exercise whose home gym has no camera coverage for that station.
6	**Automatic adherence tracking**	Member, Owner	The app cross-references the member's plan against *actually-detected* sessions (check-in + classified exercise + counted reps), not self-report. Acceptance: a "planned vs. done" weekly view shows exact matches, partial completion (e.g., planned 4 sets, detected 3), and clearly separates "not detected" from "detected but below plan volume" — because a false "you skipped it" (e.g., camera outage, played it on an uncovered machine) is a worse failure than under-crediting silently.

#	Feature	Persona	Description & Acceptance Criteria
7	**Exercise video library (curated demo set, matched to supported exercises only)**	Member	Short, side-by-side "reference vs. common mistakes" videos for exactly the exercises the engine currently scores. Acceptance: every video in the library maps 1:1 to an exercise the live-scoring system supports — no library entries for exercises the app can't yet evaluate, to avoid teaching a form the tracker can't confirm.
8	**Basic gym-operator dashboard**	Owner, Staff	Member roster, per-camera/per-station health (last-frame-received timestamp, calibration-drift warning), and today's session volume. Acceptance: a camera going dark for >2 minutes surfaces a dashboard alert and a staff push notification within that window.
9	**Subscription tier gating (technical, not just billing)**	Owner, Member	Base tier: check-in + tracking + reports + plan builder + adherence. Premium tier: adds Remote Coach/Reviewer escalation access and deeper analytics (Section 2.2/2.3). Acceptance: tier state gates feature visibility, not feature *accuracy* — every member, regardless of tier, gets an equally honest report on what was and wasn't measured.
10	**Session/plan reminders (notifications)**	Member	Push/SMS reminder for a planned session, and a same-day nudge if a planned session hasn't been detected by a configurable cutoff. Acceptance: nudges never claim a missed session with more confidence than the underlying detection actually has (e.g., must account for "camera outage" vs. "did not attend").

2.2 V1 (Sprint 3–4 scope — "pilot renewal / expansion")

#	Feature	Persona	Description
11	**Injury-risk/fatigue alerts**	Member, Coach	Surfaces the engine's set-level signals (fatigue, ROM degradation, tempo collapse, asymmetry) as trend cards, not real-time medical claims. A sustained multi-session asymmetry or ROM-degradation trend routes to the Remote Coach queue for human review, not an automatic diagnosis.
12	**Bench press support (side-camera stations only)**	Member, Owner	Enabled *only* at stations physically installed with a side-camera angle, per the engine's confirmed head-on-counting weakness. The plan builder and video library unlock bench press only for members whose home gym has a qualifying station; other gyms see it correctly greyed out with the reason stated.
13	**Progress analytics dashboard**	Member	Multi-session trend charts: joint-angle-error trend, rep-volume trend, form-score trend per exercise, streaks. Segmented so the Advanced persona (1.1C) gets raw data export while the New/Anxious persona (1.1A) gets a simplified "you're improving" summary — same data, different framing per user preference.
14	**Remote Coach review queue & tooling**	Coach, Member (Premium)	A reviewer-facing queue of flagged sessions (low-confidence classifications, escalated risk signals, premium-tier plan-customization requests) with the original clip/skeleton overlay and the engine's own confidence scores, so a human spends time only where the model is uncertain, not on every session.
15	**Gym-operator usage & business analytics**	Owner	Station utilization heatmaps by hour, class of exercise, adherence rates across the membership base, churn-risk flags (members whose detected sessions have dropped off vs. their plan).

#	Feature	Persona	Description
16	**Mirror/blind-spot install compliance tooling**	Owner, Staff	A guided camera-placement checklist tied directly to the engine's known mirror blind spot and feet-in-frame requirement (mirrors banned from any calibrated camera's field of view; a dashboard warning if a station's live feed frame includes an uncalibrated reflective surface, flagged for staff to physically re-aim or mask).

2.3 V2+ (Sprint 5+ scope — "multi-site")

#	Feature	Persona	Description
17	**Sit-up / floor-exercise counting**	Member	Ships only once the engine's hip-driven rep counter (currently a named backlog item, deliberately counting zero today) exists and is validated — the app must not promise this earlier.
18	**Multi-site operator console**	Owner (chains)	Cross-site benchmarking, fleet-wide camera health, per-site unit economics.
19	**Coach marketplace**	Coach, Member	Premium members can request a specific reviewer's ongoing oversight (paid add-on), turning the human-in-the-loop layer into incremental revenue rather than pure cost.
20	**Population-level injury-risk research view**	Owner, (opt-in)	Anonymized, aggregate trend reporting across a gym's membership for movement-pattern risk, useful to owners for equipment/programming decisions — strictly aggregate, opt-in, no individual medical claims.
21	**Self-serve gym onboarding & Jetson edge installer**	Owner	Reduces white-glove install dependency once E2 (Jetson port) lands — a guided calibration wizard replacing manual `tools/calibrate.py` runs.

---

3. Design Direction

3.1 Tone

Calibrated confidence, not hype. This product's single biggest design risk is overclaiming certainty it doesn't have — a member who's told "perfect form!" on a rep the system was actually unsure about, and then gets hurt, is the failure mode that kills trust and the business. Every screen that shows a score, a count, or an alert should carry a visible, low-friction confidence signal (not a scary disclaimer wall — a small, consistent visual language: solid outline = high-confidence 3D fused read, dashed outline = single-camera 2D estimate, greyed = not enough data). The tone is closer to a sharp, honest strength coach than a gamified fitness app — direct, specific, occasionally warm, never cutesy about safety.

Explicit anti-patterns to avoid: - No "AI is always right" theater — no fake precision (e.g., "form score: 94.7%") where the underlying error band doesn't support that precision. Round to bands or plain-language tiers instead. - No generic stock-photo gym imagery; no gradient-mesh hero sections; no illustration-heavy empty states that feel like a SaaS landing page. This product lives on a gym floor and inside a locker-room five-minute glance — it should feel like a piece of athletic equipment's UI, not a consumer app trying to look

"modern." - No dark-pattern streak-shaming or guilt notifications for missed sessions — the adherence feature's credibility depends on being scrupulously fair.

3.2 Visual Identity

- **Color palette**: charcoal/graphite base (`#1c1f22` dark surface, `#f4f2ee` warm-white light surface — avoid pure white/pure black), a single confident accent — a saturated amber (`#f5a623`) reserved *only* for actionable coaching cues and alerts, and a cool teal (`#2a9d8f`) reserved *only* for "confirmed good" states (high-confidence match to reference form). No purple/blue AI-slop gradients. Risk/injury alerts use a restrained brick red (`#c0392b`), never neon.
- **Typography**: a workhorse grotesque for numbers and data (tabular figures) for all metrics/tables, paired with a slightly more characterful display face for headlines and coaching cues, so the "human coach voice" moments feel distinct from raw data.
- **Layout philosophy**: dense-but-legible dashboard for the member's progress view and the operator console (this is a data product, not a lifestyle app); airy, single-focus screens for the in-the-moment gym-floor flows (check-in, post-set report) where the member is mid-workout and needs one clear read at arm's length, not a dashboard.
- **Visual identity signature**: use the actual skeleton/joint-angle overlay as a first-class design element throughout — not hidden in a "technical" tab. A stylized, simplified skeleton line-drawing (derived from the real pose data, not a generic icon) becomes the app's visual signature on the report screen, the video library thumbnails, and even the loading state. This is the one piece of visual identity a copycat fitness app can't fake — it's literally the product's real output.
- **Inspiration**: the data density and typographic confidence of Strava's segment/effort screens; the calm, non-gamified tone of Oura's trend charts; the physical/architectural feel of a well-designed gym equipment console (Technogym's console UI).

3.3 Key Screens & Flows

Onboarding / Check-in 1. First-time: account creation, a short liability/limitations disclosure written in plain language ("Joule currently tracks: squat, deadlift, curl, row, overhead press. It does not yet count sit-ups. Bench press is supported only at marked stations.") — an actual onboarding card, not buried legalese, because setting the coverage boundary early prevents mid-session disappointment later. 2. A one-time reference photo/short clip capture for the Re-ID embedding, taken at the check-in kiosk with clear on-screen framing guides (also where "keep your full body including feet in frame" gets taught once, visually). 3. Every subsequent visit: QR/badge tap at the kiosk to bind the visit, confirmation within 3 seconds, and a staff-assisted manual bind fallback one tap away.

Live Session View (on the member's own phone, held or set aside) - Pre-set: exercise picker (closed list, greys out unsupported items with the "why"), target sets/reps pulled from today's plan if one exists. - During set: minimal UI — a large, glanceable rep counter and the "stay in frame" cue only. Deliberately *not* a live form-accuracy overlay in MVP, because true low-latency mid-rep scoring isn't validated yet — promising live corrective feedback the pipeline can't yet deliver at rep-speed would be the single most damaging overclaim this product could make. Upgraded to true live overlay in V1/V2 once Phase D/E latency work is validated on real hardware. - Immediately post-set: the form report screen (Feature 3) slides up automatically.

Post-Session Report - Hero element: the stylized skeleton overlay for the single worst-scoring rep of the set, annotated with the specific deviation, next to the reference signature for comparison. - Below: rep-by-

rep strip (color-coded by validity/confidence), then plain-language summary, then a single clear next-action ("watch the 40-second row demo" or "logged, no action needed").

Plan Builder - Calendar-week grid, drag exercises from the supported-exercise library onto days; each exercise card shows its own confidence/coverage badge. Templates pre-populate from supported exercises only, editable.

Video Library - Grid organized by movement pattern (squat, hinge, press, pull, curl/isolation), not muscle-group marketing copy — each entry: reference clip with skeleton overlay, 2–3 common-mistake side-by-side clips, and a direct link to "try it now."

Progress Dashboard - Persona-adaptive: New/Anxious members default to a simplified "this month" summary; Advanced members can toggle a "raw data" mode exposing per-joint angle trends and CSV export.

3.4 Physical Gym-Floor Experience

- **Entrance kiosk**: a wall-mounted tablet for QR/badge check-in and the one-time reference capture only — not a full-service kiosk.
- **Station-mounted indicator, not a screen, for MVP**: a small unobtrusive light/LED strip giving the in-frame/out-of-frame cue and a "recording active" state — cheap, low-latency, doesn't compete with the phone app for attention mid-set. A full equipment-mounted display is a V2+ consideration.
- **Staff exception panel**: a lightweight front-desk tool showing today's failed check-ins, dark cameras, and any ambiguous Re-ID match — the human backstop made concrete.

---

4. Sprint Plan — from "pipeline validated" to "multi-site product"

This plan assumes the engine team's own Phase D (real-footage validation) and early Phase E (RTSP sync, Jetson port, kiosk integration) run in parallel with, and gate, specific product sprints below — they are not sequential blockers on everything, but Sprint 1's pilot cannot go live on real members until Phase D's findings are in.

Sprint 0 — Foundation (parallel with engine Phase D)

- **Goal**: de-risk the business and product assumptions while the engine team runs real-footage validation.
- **Scope**: pilot gym selection and contract; 8–12 structured interviews across all three member sub-segments plus gym owners on pricing tolerance and trust concerns; finalize the MVP exercise list against whatever Phase D actually confirms holds up on real gym footage (not just Fit3D); draft the plain-language coverage-limitations copy for onboarding; camera/station install plan for the pilot site using the existing `CAMERA_PLACEMENT.md` rules.
- **Definition of done**: signed pilot gym; Phase D findings reviewed and MVP exercise list locked; install plan approved.

Sprint 1 — Walking Skeleton

- **Goal**: prove the minimum real loop works end to end for real members at the pilot gym: check in, do a supported lift, get an honest report.

- **Scope**: Features 1–4, 7 (subset — exercises the pilot's install supports), staff exception panel. No plan builder yet — plans are staff-assisted/manual. Single-camera or minimal fused stations only.
- **Definition of done**: 20+ real check-ins by real pilot members; staff shadow-validates a sample of app reports against their own judgment; zero silent misattributions.

Sprint 2 — Plan & Adherence

- **Goal**: deliver the two pieces genuinely novel versus a plain form-feedback app — the plan builder and *verified* adherence tracking.
- **Scope**: Features 5, 6, 8, 9, 10; full curated video library for the confirmed exercise set.
- **Definition of done**: a cohort of pilot members runs a full planned week; adherence report cross-checked against a manual staff log, with a measured false-"missed"-session rate below an agreed threshold (Section 5).

Sprint 3 — Risk, Analytics, Trust

- **Goal**: move from "does the loop work" to "does this feel like a real coach, not just a rep counter."
- **Scope**: Features 11, 13, 16; refine confidence-badge visual language across all existing screens based on Sprint 1–2 member feedback.
- **Definition of done**: member survey shows form-feedback and adherence features rated as trustworthy and useful; no unresolved mirror/blind-spot install issues at the pilot site.

Sprint 4 — First Paying Pilot Gym Live

- **Goal**: convert the pilot from a validation exercise into a real, billed relationship.
- **Scope**: Feature 12 (if the pilot has a side-camera bench station), Feature 14 (Remote Coach queue, even one part-time reviewer), subscription billing turned on for real, operator dashboard (Feature 15 partial) presented to the owner as a renewal decision point.
- **Definition of done**: pilot gym owner signs a paid renewal; member subscription conversion rate and 60-day retention meet Section 5 thresholds.

Sprint 5 — Second Site & Hardening

- **Goal**: prove the product generalizes past one hand-tuned installation.
- **Scope**: full operator dashboard (Feature 15), a second pilot site with a different physical layout, RTSP live-stream support (depends on engine Phase E1), install-checklist tooling made self-guided (partial Feature 21).
- **Definition of done**: second site live with materially less white-glove engineering time than site one; cross-site camera-health monitoring from one console.

Sprint 6 — Multi-Site Product

- **Goal**: ship the features that only make sense with more than one site's worth of data and revenue.
- **Scope**: Features 17–21 as the engine's backlog items land; multi-site operator console; coach marketplace as an incremental revenue line.

- **Definition of done**: a repeatable sales/install motion for a new gym site with documented cost/timeline, and positive per-site unit economics.

---

5. Evaluation Criteria (product-level, not engine-level)

The engine's own accuracy benchmarks (RepCount MAE, Panoptic/Fit3D joint-angle error) are necessary but not sufficient — they say the pipeline is technically accurate, not that the *product* is working.

5.1 Trust & Perceived Usefulness (member-facing)

- Post-report micro-survey, sampled after a meaningful fraction of sessions. Target: sustained majority-positive by end of Sprint 3, tracked per member sub-segment separately.
- **Leading indicator**: rate at which members re-open the post-set report screen after the first glance (a proxy for "I trust this enough to read it carefully").

5.2 Adherence-Tracking Accuracy vs. Ground Truth

- During Sprint 2's pilot cohort, run a parallel manual staff log and diff it against the app's automatic adherence record. Track separately: (a) false "session missed" rate (the costlier error — must be near-zero), and (b) false "session completed" rate. A false-missed rate above a low single-digit percentage should block Sprint 4's go-live.

5.3 Engagement & Retention

- Weekly active usage of the plan/report loop among subscribed members.
- 60- and 90-day member retention for app subscribers vs. the pilot gym's own historical baseline retention — the real business case rests on this delta.
- Churn-risk flag precision (Feature 15): of members flagged as at-risk, what fraction actually cancel vs. false alarms.

5.4 Gym-Operator Willingness to Renew / Expand

- Does the pilot gym owner convert from pilot to paid (Sprint 4 gate), and does a second, *independently sourced* gym sign on (Sprint 5 gate) — the second sale is the real test of whether this generalizes.
- Qualitative: does the owner use the operator dashboard unprompted, or does it sit unused.

5.5 Unit Economics per Site

- Install/hardware cost amortized against subscription revenue at realistic attach rates.
- Ongoing cost per site: compute (edge vs. cloud inference cost per member-session), Remote Coach staffing cost per flagged session, support/staff-exception load.
- Target: a defined payback period on install cost, validated with Sprint 4's real pilot numbers rather than assumed.

5.6 Safety & Failure-Mode Rate

- **Mirror/phantom-duplicate incident rate**: any real installation event where a mirror produced a duplicate tracked person or contaminated a real member's reading, post-install-checklist. Target near-zero.
- **Out-of-coverage silent failure rate**: any case where a member attempted an unsupported exercise and the app produced a confident- looking but wrong read instead of correctly declining/flagging it. Must be effectively zero — a design/gating bug, not an accuracy limitation.
- **Injury-risk alert precision/recall**, once enough Sprint 3+ data exists — always framed as directional, never clinical or medical-grade.

---

6. Honest Scope Boundary (restated as a single source of truth)

Because this is the section most likely to be quietly eroded under launch-date pressure, it is restated plainly here:

- **Ships at MVP**: squat, deadlift, bicep curl, row, overhead press, lateral raise — standing/unencumbered movements, the set the engine has actually validated cleanly. Squat ships with a visibly wider confidence band than the others, not silently equal.
- **Does not ship at MVP, and the app must say so, not hide it**: sit-ups (engine counts zero today), bench press at any station without a physically installed side camera, any exercise where a mirror is in a calibrated camera's field of view (excluded by install rule, not detection), any session where the member's feet leave frame (flagged in real time via the station indicator, not silently mis-scored).
- **Real-time is redefined honestly as near-real-time**: mid-rep live-overlay coaching is a V2+ feature gated on Phase E latency work; MVP's "real-time feedback" promise is a same-set, immediately-post-rep report — still meaningfully faster and more specific than a human trainer glancing over occasionally.

PART 2

Technical Roadmap

Phase F onward: what has to be built under the app — persistence, identity, privacy, a live floor runtime, backend, and multi-tenant scale-out — grounded line-by-line in the current codebase.

Joule — Productization Roadmap: Phase F Onward

Written 2026-08-06. **Status of this document: planning only.** Nothing below has been built. It continues the phase-numbering convention of `ROADMAP.md` (Phases A–E) and assumes Phases D and E are the prerequisite gates they are described as there. It is a companion document to `PRODUCT_SPEC.md` (the member/owner-facing product specification for the same plan).

Ground rule applied throughout: every claim about "what exists today" cites a file actually read in this codebase. Where the repo is silent, that is said explicitly rather than assumed.

Part 0 — Verified current state (the baseline this plan builds from)

This section exists so the gap analysis can't drift into fiction. Each line is a fact traced to code or docs.

0.1 What the system is, structurally

`pipeline.py` is an argparse CLI. It builds one `CameraProcessor` (`src/camera.py`) per `--sources` entry, each in a `threading.Thread`, joins them all, then calls `write_merged_json`. The process lifetime *is* the session lifetime. There is no service mode, no supervisor, no scheduler, no restart path, no daemon.

`--fuse rig.npz` swaps in a single `FusedCameraProcessor` (`src/fusion.py`) that reads all N sources in lockstep inside one thread and runs detect+pose for every view sequentially in that same thread.

0.2 Where state lives

All per-person exercise state lives in a plain dict (`persons_data`) local to `CameraProcessor.run()`. Angle history accumulates unbounded — `pd['angles'][joint].append((timestamp, val))` runs per joint, per tracked person, per frame. Nine joints × 30 fps means roughly a million tuples per person per camera per hour, held in RAM for the life of the process.

Results are serialized exactly once, at the end of `run()`, via `write_camera_json`. A crash mid-session loses the entire session. `Ctrl+C` / `stop_event` does write partials, but an OOM, a segfault in a native dependency, or a power cut does not.

0.3 The clock problem

Analysis timestamps are `timestamp = frame_idx / fps` — a source-relative clock that starts at `0.0` (`src/camera.py`, identically in `src/fusion.py`). Every rep timestamp, form window, and angle series in the output JSON is on that clock. There is no wall-clock time, no date, and no session identifier anywhere in the output.

Separately, `src/session.py` uses `time.time()` for `Person.last_seen` and staleness. Identity and analysis already run on two different, unrelated clocks.

0.4 Identity, precisely as built

`SessionGallery` (`src/session.py`) is an in-memory `dict[str, Person]`, constructed fresh in `pipeline.py` on every invocation, with `self._counter` restarting at zero — so `person_1` means a different human on every run. Embeddings are never written to disk; `README.md` states this as a deliberate property ("Embeddings live only in memory for the session — nothing biometric is persisted") and `DEVELOPMENT.md` repeats it as a core design decision.

The embedding itself (`src/reid.py`) is MobileNetV3-small ImageNet features (576-d, `_DEEP_WEIGHT = 0.6`) concatenated with an HSV histogram of the torso band. It is not a Re-ID-trained model, and by construction it is heavily clothing-driven.

`REID_STALE_SECONDS = 1800` (30 min, `src/config.py`) excludes anyone not seen recently from match candidacy. So even *within* one long run, a member who lifts, leaves for 40 minutes, and returns enrolls as a brand-new person.

`/checkin?person=person_1&name=Ahmed` calls `bind_name`, which sets a string on the in-memory `Person`. That binding dies with the process.

0.5 The HTTP surface

`src/dashboard.py` runs a `http.server.BaseHTTPRequestHandler` bound to `0.0.0.0`. It exposes: `GET /` (hardcoded HTML), `GET /snapshot/<cam_id>` (live JPEG of the gym floor), `GET /data` (current reps/angles/state), `GET /people` (every tracked identity), `GET /checkin?person=…&name=…` (binds a member name). There is no authentication, no authorization, no TLS, no rate limiting, and no audit log on any of these. `/checkin` is a state-mutating GET.

`push_data` is keyed by `cam_id` alone, and `src/camera.py` explicitly pushes only "the person with the most frames"; `src/fusion.py` picks the largest bounding box. **The dashboard is structurally single-person-per-camera.** It is an operator debug view, not a product surface.

Doc inconsistency worth noting: `README.md` documents `/people` and `/checkin` on port 8080, but the dashboard's default is `--port 5000`.

0.6 What the system can actually count

Rep counting is driven by exactly two signals (`src/camera.py`, mirrored in `src/fusion.py`): the knee average → `squat_sm`, and the elbow average → `upper_sm`. There is no third counter.

`CLASS_FAMILY` (`src/exercise_classifier.py`) routes `situp` and `cardio` to family `'none'` — deliberately zero reps. `CLASSES` lists 12 labels, but per `FIT3D_UTILIZATION_PLAN.md` and the Phase B1 verification in `ROADMAP.md`, the trained model in `models/exercise_clf.npz` only ever emits 7 (`cardio, press, pullup, pushup, raise, situp, squat`). `hinge`, `lunge`, `curl`, `row` are heuristic-only and never appear as trained output.

0.7 What "form score" actually measures

`src/form.py` starts at 100 and deducts on five rule-based checks. Its own docstring says scores are "relative to the person's own set where possible (median-based)." The depth check keys off `rep['valid']`, which `src/state_machine.py` sets as `lowest <= self.depth_threshold`, where `depth_threshold = lo + DEPTH_FRACTION * rng` over the person's *own last 3 seconds* of motion (`DEPTH_FRACTION = 0.25`).

Today's form score is therefore largely self-referential, not a measure of correctness against a standard. The absolute path exists (`src/signature.py` + reference signatures), but `models/` contains only `exercise_clf.npz` — there is no `models/references/` directory in the repo, and `ROADMAP.md` D3 records that the only reference test so far scored a reference against its own source video.

0.8 Measured performance, and what is unmeasured

`FIT3D_UTILIZATION_PLAN.md` records the only real throughput number in the repo: **~0.22 s of wall-clock processing per video frame** for a 4-camera fused run (detection + pose + triangulation), on the development laptop, with a single subject in frame. That is roughly 4.5 processed frames/second aggregate, ~1.1 fps per view, for one person.

Pose is top-down with `num_poses=1` (`src/pose.py`) — one MediaPipe `detect()` call per detected person per frame. **Cost scales linearly with the number of people in frame.**

MediaPipe is forced to the CPU delegate for a macOS-specific PyTorch/Metal conflict. `README.md` says "Not an issue on Jetson," but `ROADMAP.md` E2 is explicit that Jetson is "currently entirely unmeasured on target hardware." `DEVELOPMENT.md`'s "2–4 cameras per Orin Nano at reduced frame rates" is a stated estimate, not a measurement.

`src/config.py` records measured detection rates on heavily-occluded two-person footage: **yolov8n@640 detected both people 30% of the time** (yolov8s@640 51%, yolov8s@960 61%). The shipped default is yolov8n@640.

0.9 What is absent entirely

No database, no ORM, no web framework beyond `http.server`. `requirements.txt` is six lines (opencv, ultralytics, mediapipe, numpy, lapx, torchvision). No `tests/` directory (`ROADMAP.md` A4 states outright that "no `tests/` convention exists in this repo"). No `.github/` CI. No Dockerfile. No deployment manifests. No per-environment or per-site configuration mechanism — `src/config.py` is module-level constants imported at import time, global to the process.

0.10 Two pieces of live technical debt worth naming now

(a) Per-joint timestamp drift in the output format. `_downsample_series` in `src/output.py` runs an independent sampling clock per joint, keyed off that joint's own emitted timestamps:

```
next_t = entries[0][0]
for ts, val in entries:
    if ts >= next_t - 1e-9:
        sampled.append(...)
        next_t = ts + interval
```

`ROADMAP.md` C3 and `HANDOVER6.md` document that one transient single-frame confidence gap in one joint permanently shifts that joint's output timestamps off the shared frame grid. The Fit3D validation tool was fixed to score joints independently; **`src/output.py` itself was not changed.** Any future consumer — a mobile app chart, an analytics warehouse, a per-frame join in a DB — that buckets joints on a shared timestamp will reproduce exactly the artifact that nearly got reported as a pipeline bug.

(b) Detector licensing. `requirements.txt` pins `ultralytics>=8.2.0` and `DETECTOR_MODEL = 'yolov8n.pt'`. Ultralytics YOLOv8 ships under AGPL-3.0 with a separate commercial license offering. This needs counsel review before anything ships commercially — flagged as an open legal question, not a settled fact.

Part 1 — Gap analysis: "validated pipeline" → "a gym can sell this"

Lift sizing: **S** = days, **M** = weeks, **L** = a quarter-ish with a small team, **XL** = the thing that decides whether this is a product at all.

1.1 The four biggest lifts, called out first

#	Gap	Lift	Why it's the hard one
1	**Real-gym accuracy under crowding, occlusion, mirrors and equipment**	**XL**	Entirely unknown. Phase D has never run. The strongest adverse signal in the repo: the shipped detector caught both people only 30% of the time on heavily-occluded two-person footage (`src/config.py`). A commercial gym floor is dozens of people, machines, and mirror walls.
2	**Throughput on affordable edge hardware at real gym density**	**XL**	0.22 s/frame for 4 views with *one* person, on a laptop. Pose cost is linear in people (`num_poses=1`, one call per person). A 15-person camera view is a different order of magnitude. Jetson numbers do not exist.
3	**Persistent per-member identity that holds for a 90-minute visit on a busy floor**	**XL**	Not "persist the gallery." The embedding is ImageNet features + clothing histogram; it will not match the same person next Tuesday in different clothes. The real problem is holding a *check-in-established* binding across an entire visit, through fragmentation, equipment occlusion, and 40 other similarly-dressed people, when `REID_STALE_SECONDS = 1800` already drops anyone gone 30 minutes. Mis-attribution is called "the worst failure mode" in `src/session.py` — as a product it also becomes a trust and billing failure.
4	**Privacy, consent, and biometric-regulation compliance**	**L, and non-negotiable**	The current system is legally clean precisely *because* it persists nothing. Every gap below that fixes a product problem — persistent identity, historical storage, video retention — actively destroys that property. This is the only gap where doing the engineering work *creates* the legal exposure.

1.2 Multi-tenancy and per-gym data isolation

Exists: nothing. No concept of a gym, an organization, an account, or a tenant anywhere in the codebase. `--names squat_rack_1` is a station string used as a JSON filename.

Needed: a tenant as a first-class entity threaded through storage, API authorization, model/config scoping, and the edge runtime's identity. Camera calibration (`rig.npz`, today a local file path) and per-site thresholds must become tenant-scoped managed records, not files on someone's laptop.

Non-obvious sub-gap: `src/config.py` is global module state. `DETECTOR_CONF`, `REID_MATCH_THRESHOLD`, `REID_MATCH_MARGIN`, `TRUNK_LEAN_LIMIT`, `MIN_ANGLE_RANGE` will all need per-site tuning (lighting, ceiling heights, clientele, equipment). Today changing any of them is a code edit affecting every deployment simultaneously. **Lift: M for config layering, L for full tenant isolation.**

1.3 Persistent per-member identity

Exists: in-memory session gallery, `/checkin` name binding that dies with the process, `person_N` counter restarting each run.

Needed: a durable `member_id`, a `visit` concept, and a binding between the two that survives process restarts and is auditable.

Architectural warning, grounded in the code: do not solve this by persisting `SessionGallery` embeddings across visits. Two independent reasons. Technically, the embedding is ImageNet-pretrained generic features plus a clothing-color histogram, designed for *within-session* matching, where `src/reid.py`'s own docstring assumes "clothing stays constant." Legally, a persisted appearance template derived from a person's body is precisely what biometric-identifier statutes are written about; a session-scoped, never-written embedding is a far weaker regulatory claim than a stored gallery. **Lift: L, and it is lift #3 above.**

1.4 Long-term historical storage

Exists: one JSON file per station per run, written once at process exit, into whatever `--output` directory the operator typed. No session ID, no timestamp in the filename — a second run into the same directory silently overwrites the first.

Needed: a real database, plus a deliberate split of hot and cold data. `OUTPUT_FPS = 30.0` means `angle_series` is emitted at full frame rate, ~9 joints × 30 Hz × session duration, per person. Reps and sets are small, relational, and queried constantly; angle series are large, append-only, and queried rarely. Putting both in the same store is a design error that will be very expensive to undo. **Lift: M for schema/ingest, L including migration/retention tooling.**

1.5 A real backend API

Exists: `src/dashboard.py`'s five unauthenticated `http.server` endpoints.

Needed: an authenticated, versioned, documented API with tenant-scoped authorization on every route; a separate ingest path for edge→cloud event submission (device credentials, not user credentials); idempotency, since edge devices will retry after network loss. **Lift: M** — conventional, but everything else depends on it.

1.6 Mobile/web app backend needs

Exists: nothing. No user accounts, no auth, no sessions, no push infrastructure.

Needed: account lifecycle, member↔gym membership records, auth (and the app's check-in flow doubling as the identity-binding mechanism per 1.3), a read model shaped for app screens rather than for the pipeline's convenience, offline tolerance, real-time delivery if in-workout feedback is promised. **Lift: L** (large but low-risk).

1.7 Plan builder and adherence tracking

Exists: nothing. No plan, no schedule, no target, no goal concept.

Needed: a plan model (template → assigned plan → scheduled session → prescribed exercise/sets/reps/load) and an adherence engine that reconciles *prescribed* against *observed*.

The hard part is not the schema — it's the epistemics. Adherence is a claim about what a member *didn't* do. The system's own documented limitations mean non-detection is routinely not evidence of non-performance: sit-ups deliberately count zero (`CLASS_FAMILY['situp'] = 'none'`), bench press counting is weak head-on, any station without camera coverage produces nothing, any set where feet left frame degrades silently. An adherence feature that reports "you skipped your prescribed sets" on that basis will

generate false accusations against paying members. The data model must distinguish **observed-complete / observed-partial / not-observable / observed- absent**, and only the last may ever drive a negative statement. **Lift: M for the model; the observability-classification design is genuinely subtle.**

1.8 Exercise video content library and delivery

Exists: nothing. `--visualise` writes annotated MP4s of *members* for debugging; that is not a content library.

Needed: demo video production or licensing, a catalog keyed to the same exercise taxonomy the classifier uses, transcoding/ABR, CDN delivery, offline caching, captions/accessibility, localization if multi-market. Taxonomy trap: the library will want dozens of exercises; the classifier recognizes 7 trained classes. The catalog must be able to contain exercises the pipeline cannot detect, clearly marked as such. **Lift: S–M engineering; content production is a separate budget line.**

1.9 Notifications and reminders

Exists: nothing.

Needed: push infrastructure, per-user preferences and quiet hours, a scheduling/trigger engine, delivery-attempt audit trail. Any notification derived from adherence inherits 1.7's false-negative risk, now delivered to the member's lock screen; any notification with form feedback is health-adjacent personal data leaving the platform. **Lift: S–M.**

1.10 Billing and subscriptions

Exists: nothing.

Needed: likely two tiers — B2B (gym pays per site/camera/member) and B2C (member subscription, possibly gym-collected). Plus proration, dunning, tax, entitlement checks, and a clean answer to "the cameras were down for three days." **Lift: M** (buy, don't build the processor).

1.11 Gym-operator admin dashboard

Exists: `src/dashboard.py` — one card per camera showing one person, no auth, no history.

Needed: floor/camera health, per-camera coverage and drift status, member roster and check-in state, aggregate utilization (often what the operator actually buys), incident/alert view, consent/retention administration. Must be role-scoped — gym staff should not have unrestricted access to every member's body-mechanics history. **Lift: M–L.**

1.12 Camera install and fleet operations across sites

Exists: `CAMERA_PLACEMENT.md` (evidence-traced install guide), `CAPTURE_PROTOCOL.md`, `tools/calibrate.py` for a two-camera checkerboard pass producing a local `.npz`.

Needed: a site-survey workflow, an install/commissioning app that validates each camera against the `CAMERA_PLACEMENT.md` rules *at install time* (feet-in-frame check, mirror check, spacing check), calibration as a managed versioned artifact rather than a file, and — critically — **automatic calibration-drift detection.** `CAMERA_PLACEMENT.md` rule 7 states plainly that a bumped camera "silently invalidates that rig — the pipeline

has no way to detect a moved camera on its own." Also needed: remote device fleet management (inventory, OTA updates, staged rollout, rollback, remote logs). **Lift: L, and consistently underestimated.**

1.13 Model-serving and inference infrastructure

Exists: models loaded in-process per thread.

Needed: measured capacity on real target hardware (Phase E2, never run), a decision on GPU delegate / TensorRT / quantization for Jetson, an answer to the linear-in-people pose cost (batch crops, or a bottom-up multi-person model), frame-drop and back-pressure policy under load, model versioning with per-site pinning and rollback.

Also needed: a licensing decision on the training data. `DEVELOPMENT.md` states Fit3D and M3GYM are "academic-only licenses — validation OK, shipped-model training not." The currently-shipped `models/exercise_clf.npz` was trained on the RepCount train split (Phase 9); RepCount came from a public HF mirror with no access blocker, but its terms need confirming before commercial deployment. **Lift: XL (this is lift #2).**

1.14 Privacy, consent, and compliance

Exists: exactly one relevant property, and it's a good one — nothing biometric is persisted, embeddings are session-only, no video is retained by default.

Needed, and each item erodes that property:

- **Consent capture and withdrawal** — per member, versioned, timestamped, revocable, enforced downstream. Plus the *bystander* problem: guests, staff, cleaners, contractors, minors who appear in frame and never consented. The current pipeline silently enrolls anyone above `DETECTOR_CONF` as `person_N`.
- **Video retention and deletion policy** — default should be that raw frames never leave the edge device and are never written. Any clip retention offered (dispute resolution) needs explicit opt-in, a hard short TTL, encryption at rest, access logging.
- **Biometric-specific statutes.** BIPA (Illinois) requires written notice and consent before collecting a biometric identifier, a published retention/destruction schedule, and carries a private right of action — the single most expensive US exposure. Texas CUBI and Washington's biometric law are similar. GDPR/UK GDPR treat biometric data processed to uniquely identify a person as special-category (Art. 9), needing an Art. 9 condition on top of a lawful basis; systematic large-scale monitoring of a publicly accessible area triggers a mandatory DPIA (Art. 35). Children's-data regimes apply if minors are members.
- **Data-subject rights** — access, portability, erasure, including erasure propagating to derived analytics and any model trained on member footage. If member footage ever feeds `training_data/` (`ROADMAP.md` D4 proposes exactly this), erasure becomes a model-retraining obligation.
- **Health-data positioning.** `src/risk.py` emits fatigue, ROM-degradation, tempo-collapse and asymmetry signals with a `risk_level`. `STATUS.md` and `src/risk.py` both state these are training-load indicators, not medical claims — that framing must survive contact with marketing and terms of service.
- **Signage, transparency, and a DPIA/PIA** per site.

Lift: L, and it gates go-live — this cannot be retrofitted.

1.15 Monitoring, alerting, and on-call

Exists: `logging.basicConfig(level=logging.INFO)` to stdout, a progress log every 300 frames, some `logger.warning` calls on lost tracks and cross-view inconsistency.

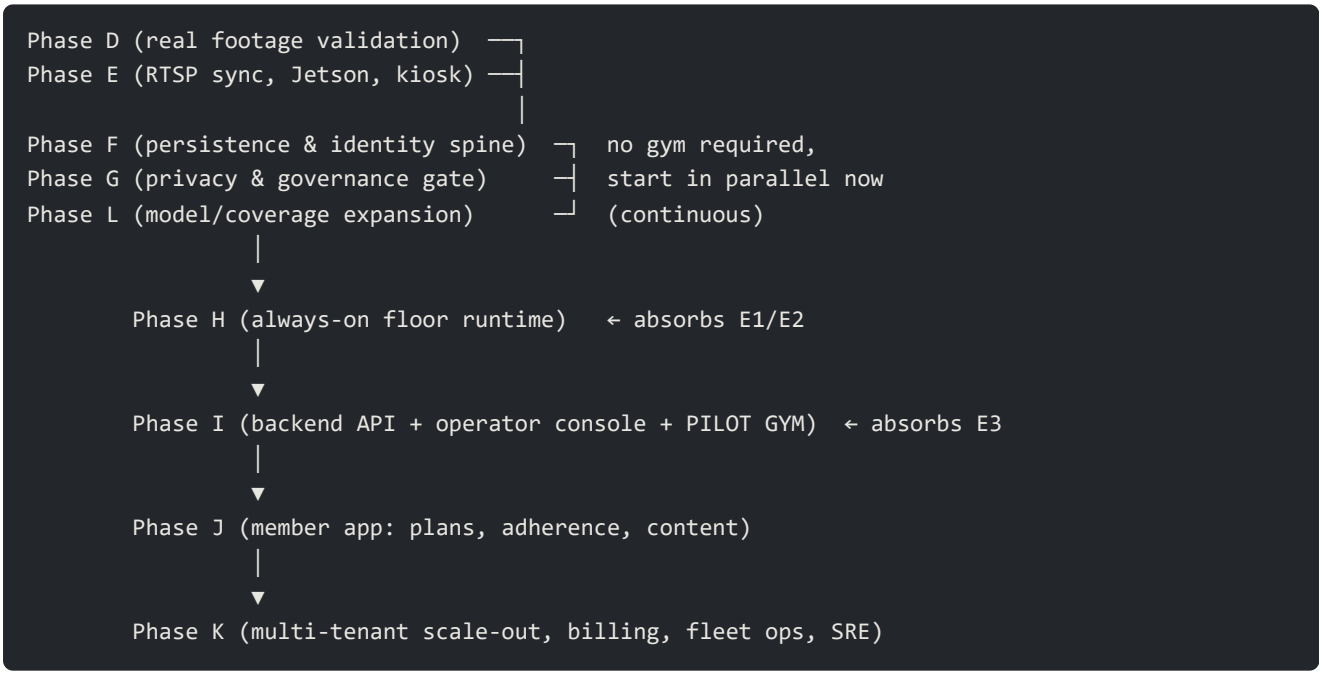
Needed: structured logs shipped off-device; metrics (frames processed, dropped, detections/frame, identity-resolution rate, Re-ID enrollment rate, triangulation-fallback rate, per-stage latency); product-quality alerts, not just liveness (camera dark, camera moved, identity fragmentation spiking, a station producing zero reps during business hours); SLOs; incident response and on-call; a status/comms path for operators. **Lift: M–L.**

1.16 Engineering hygiene prerequisites

No automated test suite (validation scripts for A1–A4 were deliberately kept in scratchpads and never committed, per `ROADMAP.md`), no CI, no reproducible packaging/containerization, no secrets management, no environments (dev/staging/prod), no release process. **Lift: M.**

Part 2 — Phased engineering roadmap

Sequencing principle, consistent with how Phases A–D were reasoned about in `ROADMAP.md`: **do the work that has no external dependency first, and do not spend an irreplaceable input (a pilot gym's goodwill and its members' trust) on foreseeable failures.** A failed pilot is more expensive than a delayed one.



Phase G is drawn parallel to F but is a **hard gate on H**: no real member's body may be processed under a commercial arrangement before G's minimum set lands.

Phase F — Persistence and identity spine

Goal: make one pipeline run durable, wall-clocked, crash-survivable, and attributable to a stable member — without changing any CV behaviour.

Why here: every later phase reads from this. It needs no gym, no hardware, and no footage — it can start today against `test_videos/` and Fit3D clips. Doing it before the pilot means the pilot generates real data instead of throwaway JSON.

Prerequisites: none.

- **F1. Wall-clock time base.** Add an absolute session start time and carry `wall_time = session_start + frame_idx/fps` alongside the existing relative clock. Do not remove the relative clock — `src/state_machine.py`, `src/form.py`, and the validation tooling all depend on it. Reconcile with `src/session.py`'s independent `time.time()` usage.
- **F2. Session, visit, and person as durable records.** Introduce `session_id`, `visit_id`, `member_id`. Stop deriving output filenames from `--names` alone (today a re-run silently overwrites).
- **F3. Incremental event emission.** Replace "one JSON at process exit" with an append-only event stream: `person_observed`, `identity_bound`, `set_started`, `rep_completed`, `set_ended`, `risk_assessed`, `camera_health`. Write locally first (durable spool), ship second. The single most valuable structural change in the phase — decouples analysis from delivery and makes crash recovery possible.
- **F4. Bounded memory.** Cap or window the in-RAM `pd['angles']` accumulation so an 8-hour run doesn't grow monotonically. `src/form.py` and the classifier only ever read trailing windows (`CLASSIFY_WINDOW_SECONDS = 6.0`, per-rep `[t0, t1]` windows), so a bounded ring buffer plus streamed-out cold history is behaviour-preserving. **Verify against `src/signature.py` and `src/risk.py` before implementing** — `assess_track` operates over the whole set.
- **F5. Storage layer.** Relational store for members, visits, sessions, sets, reps, form scores, risk assessments (small, queried constantly). Separate object/time-series store for angle series (large, append-only, rarely read). Tenant column on every table from day one.
- **F6. Fix the per-joint downsample drift.** `_downsample_series` in `src/output.py` — emit on a shared frame grid with explicit nulls, rather than a per-joint independent clock. This bug already produced a false finding once (`ROADMAP.md` C3). Re-run Panoptic and Fit3D validations afterward to confirm error figures are unchanged.
- **F7. Identity binding model.** Two layers, explicitly separated:
- *Durable layer:* `member_id`, established by an **authenticated check-in event** (QR scan, app NFC/BLE, kiosk tap), not by appearance.
- *Ephemeral layer:* the existing `SessionGallery`, rescoped to a visit, binding tracklets → `visit_person`. Add a `bind_visit_person_to_member(visit_person_id, member_id, evidence)` operation recording *how* the binding was made and its confidence.
- **Persist no embeddings across visits.** Both the technically honest choice (clothing-dominated embedding) and the compliance-cheap one.
- Make **unattributed people first-class**: guests, staff, and non-consenting bystanders need a defined disposition.
- **F8. Test and CI foundation.** Create `tests/`. Recover the A1–A4 validation scripts from scratchpad history if possible, or rewrite them. Add CI running unit tests plus a smoke run of `pipeline.py`.

Exit criteria: 1. A pipeline run killed with `SIGKILL` at minute 20 of a 30-minute clip loses at most the last N seconds of data; everything before is queryable from the database. 2. Two runs against the same output target no longer overwrite each other; every rep is retrievable by `(member_id, visit_id, timestamp)` in absolute wall-clock terms. 3. An 8-hour continuous run shows flat, not growing, resident memory. 4.

Panoptic and Fit3D validation figures re-run post-F6 are unchanged within noise (proving F didn't touch CV behaviour). 5. `tests/` exists, CI is green, and the A1/A2/A3 identity/fusion assertions run automatically.

Phase G — Privacy, consent, and data governance gate

Goal: establish the legal and ethical envelope *before* any real member's body is processed under a commercial arrangement.

Why here: the current system's compliance posture is good only because it stores nothing. Phase F deliberately destroys that. G must close behind it — retrofitting consent onto collected data is not possible.

Prerequisites: F5/F7 designs (not necessarily their implementation).

- **G1. Data inventory and flow map.** Every category the system produces (raw frames, crops, embeddings, keypoints, joint angles, reps, form scores, risk signals, names): where produced, stored, how long, who can read it, how it's deleted.
- **G2. Legal review and jurisdiction scoping.** Which markets. Whether appearance embeddings and skeletal keypoints constitute biometric identifiers in each. GDPR Art. 9 analysis and lawful basis. DPIA for the pilot site. BIPA/CUBI applicability if US. Minors policy.
- **G3. Consent lifecycle, engineered.** Versioned consent records; a machine-readable consent state the edge runtime enforces at processing time (not just at reporting time); working withdrawal that stops future processing and triggers deletion; a bystander/non-member policy with signage.
- **G4. Retention and deletion, implemented and tested.** Default: **raw video never persisted, never leaves the edge device.** Per-category TTLs. An erasure job that provably removes a member from the relational store, angle-series store, backups, logs, and any derived training set. Test it by actually deleting a test member and auditing.
- **G5. Access control and audit.** Role-scoped access — gym staff, member, support, engineer. Every access to member-level body data logged. Gym staff must not have unrestricted access to every member's biomechanics history.
- **G6. Kill the unauthenticated surface.** `src/dashboard.py` must not exist in any deployment, even a "temporary" pilot. Either bind loopback-only behind an authenticated proxy, or replace it outright in Phase I.
- **G7. Product-claim boundaries.** Written, agreed with legal, binding on marketing: `src/risk.py` outputs are training-load indicators, not injury prediction or medical advice; form scores are guidance, not certification; the supported-exercise list is explicit; adherence is evidence-of-observation, never evidence-of-absence.
- **G8. Training-data provenance policy.** `ROADMAP.md` D4 proposes feeding real footage back into `training_data/` — the right ML instinct and a compliance landmine. Decide now: separate consent for training use, opt-out honoured, a documented position on erasure vs. a trained model. Confirm the license position on `models/exercise_clf.npz`.

Exit criteria: 1. Signed-off DPIA/PIA for the pilot site, and a written legal opinion on biometric-identifier status in the target jurisdiction. 2. Consent capture, enforcement, and withdrawal work end-to-end on a test member, demonstrated. 3. An erasure request removes a test member from every store including backups and derived sets, verified by audit query. 4. No unauthenticated network endpoint exposes member data or gym-floor imagery in any environment. 5. Retention TTLs enforced by an automated job, not policy documents. 6. Public-facing claims reviewed against G7 and the `STATUS.md` limitations list.

Phase H — The always-on floor runtime

Goal: convert the CLI batch script into a supervised, long-running, self-healing service that runs a gym floor for a full trading day unattended, on hardware someone will actually buy.

Why here: the pilot cannot happen without it. Absorbs `ROADMAP.md` E1 (RTSP timestamp sync) and E2 (Jetson port and throughput).

Prerequisites: Phase D findings, F1–F4, E1/E2.

- **H1. Service lifecycle.** Replace "start threads → join → write JSON → exit" with a supervised process: config-driven camera set, health endpoint, graceful reload, per-camera restart on failure without killing the floor, a spool that survives restarts.
- **H2. RTSP ingest and timestamp sync (E1).** `src/fusion.py`'s docstring is explicit: "Live RTSP streams need timestamp alignment upstream; not handled here yet." A single dropped frame on one live stream permanently desynchronizes a fused station with no error raised. Needs real timestamp-based alignment, reconnection with backoff, and a defined behaviour when a view falls behind.
- **H3. Jetson port and honest capacity measurement (E2).** Revisit the CPU-delegate forcing in `src/pose.py` (its rationale is a macOS PyTorch/Metal conflict, irrelevant on Jetson). Evaluate GPU delegate, TensorRT, quantization. Then **measure, do not estimate**: frames/ second/camera at 1, 3, 5, 10, 15 people in frame; the resulting cameras-per-device number; the accuracy cost of every throughput knob.
- **H4. Address the linear-in-people pose cost.** Dominant scaling term on a busy floor. Evaluate batching crops through one inference call, reducing pose rate for people not in an active set, and a bottom-up multi-person model. A genuine research-flavoured sub-project.
- **H5. Back-pressure and degradation policy.** Define what the system drops first when it can't keep up, and make the choice explicit and observable: frame rate before accuracy, 3D fusion before 2D counting, visualization before analysis.
- **H6. Per-site and per-camera configuration.** Break `src/config.py`'s global constants into a layered resolution (defaults → tenant → site → camera), loaded at runtime.
- **H7. Calibration as a managed artifact + drift detection.** Move rig calibration off local `.npz` paths into versioned, site-scoped records. A candidate signal: a sustained rise in `cross_view_consistent()` rejection rate at a fused station is exactly what a bumped camera looks like, and `src/fusion.py` already logs that event.
- **H8. Observability.** Structured logs off-device; the metrics listed in 1.15; per-camera health heartbeat.

Exit criteria: 1. Eight continuous hours of unattended operation on target hardware against live RTSP feeds, with a documented, measured cameras-per-device figure at realistic occupancy. 2. Pulling one camera's network cable: that camera recovers automatically, others unaffected, the gap is visible in monitoring. 3. Fused-station accuracy on live RTSP within a stated tolerance of the same scene from pre-synchronized recordings. 4. Killing the service mid-day loses at most N seconds and auto-resumes. 5. Nudging a calibrated camera raises a drift alert without a human noticing first. 6. Back-pressure behaviour under 2× overload is documented, observable, matches the H5 policy.

Phase I — Backend API, operator console, and the pilot gym

Goal: one real gym, real members, real money-adjacent operation. The first honest test of the product thesis.

Prerequisites: F, G (mandatory), H. Absorbs E3 (kiosk/QR check-in).

- **I1. Control-plane API.** Authenticated, versioned, tenant-scoped. Device credentials for edge event ingest (idempotent, retry-safe, replay-resistant); user credentials for reads.
- **I2. Check-in as the identity anchor (E3).** Replace the current GET endpoint with an authenticated flow producing a signed check-in event (`member_id` , timestamp, location). Solve the association problem: which `visit_person` is this member? Options, increasing robustness — a spatial+temporal gate, member-confirmed in-app disambiguation, dedicated entry-camera framing. **This binding is lift #3 and deserves dedicated design time.** Define and measure a mis-binding rate; treat mis-binding as a P1 incident class.
- **I3. Operator console.** Replaces `src/dashboard.py` : floor/camera health, calibration/drift status, check-in state, aggregate utilization, incident view, consent administration, role-scoped access.
- **I4. Member read API.** Session history, sets, reps, form scores, trends, shaped for app screens.
- **I5. Ground-truth accuracy audit.** The most important deliverable of this phase. Human observers manually count and label a statistically meaningful sample of real member sets at the pilot site; compare against system output; publish per-exercise, per-station, per-camera- configuration precision/recall. **This produces the number the product is allowed to promise.**
- **I6. Support and incident process.** A member says "these aren't my reps" — who can look, what can they see, how is it corrected and audited?

Exit criteria: 1. One gym live for 30 consecutive operating days. 2. A published, honest, per-exercise accuracy table from I5's human-ground-truth audit, including the exercises/conditions where the system does poorly. 3. Measured identity mis-attribution rate below an explicitly stated, pre-committed threshold. 4. Zero unauthenticated data-bearing endpoints; a passing external security review. 5. Gym operator can run the site day-to-day without engineering intervention. 6. A member exercises data access and erasure, end-to-end, successfully.

Phase J — Member app: plans, adherence, and content

Goal: the founder's product vision, made real — automatic tracking, right/wrong feedback with a score, a plan built in-app, adherence against it, a demo-video library.

Why here: depends on I5's accuracy table. Building plan/adherence UX before knowing the true per-exercise accuracy risks designing a feature the pipeline cannot honestly support.

Prerequisites: Phase I, especially I5.

- **J1. App foundation.** Accounts, membership, auth, onboarding, offline tolerance.
- **J2. Automatic session view.** Sets, reps, per-rep form, trends across visits — must render *confidence and coverage*, not just numbers.
- **J3. Form feedback and the accuracy score.** Today's `src/form.py` score is self-referential (Part 0.7). Two honest options: present it as "consistency and control within your set," or complete the reference-signature path (`src/signature.py` , `tools/build_reference.py` , `ROADMAP.md` D3) for an absolute standard. **The second is required if the product promises "are you doing it right."** `src/config.py` itself warns that a reference from a different camera angle "reads perspective differences as form deviations" — real content-and-validation work, not a code change.
- **J4. Plan builder.** Templates and custom plans, exercise catalog aligned to the classifier taxonomy, unsupported-but-catalogued exercises clearly marked as self-logged.

- **J5. Adherence engine.** Reconcile prescribed vs observed using the four-state observability model from 1.7. Only `observed-absent` may drive a negative statement. Manual self-logging as the escape hatch for everything cameras can't see — a core part of an honest product, not a fallback.
- **J6. Content library.** Demo videos per catalog exercise, transcode/CDN/offline cache, deep-linked from form cues (`src/form.py`'s `CORRECTIVE_CUES` already map naturally onto "watch how").
- **J7. Notifications.** Reminders, session summaries, plan nudges, with quiet hours/preferences; adherence-derived messaging held to J5's evidentiary standard.
- **J8. Real-time vs per-session feedback decision.** Per-session is achievable now. Real-time in-workout coaching adds a latency budget, a delivery channel, and a UX question — treat as a separate, later, explicitly-scoped increment.

Exit criteria: 1. A pilot member can build a plan, train, and see accurate reps, form feedback, and adherence for that plan the same day. 2. Adherence never asserts non-performance for an exercise/station the system could not observe — verified by adversarial testing. 3. The form/accuracy score is either backed by a validated reference signature or worded to match what it actually measures. 4. Every catalog exercise has a demo video; every camera-untrackable exercise is visibly marked as self-logged. 5. Measured member retention/engagement over a defined pilot window, plus qualitative feedback on whether the coaching is genuinely useful.

Phase K — Multi-tenant scale-out

Goal: gyms two through N, without engineering being in the loop for each one.

Prerequisites: Phase I proven at one site, Phase J shipped.

- **K1. True multi-tenancy.** Tenant isolation enforced and *tested* (a cross-tenant read attempt must fail, in an automated test). Per-tenant configuration, models, catalogs, branding.
- **K2. Billing and subscriptions.** Per 1.10, including service-credit policy for camera downtime.
- **K3. Fleet operations.** Device inventory, OTA updates with staged rollout/rollback, remote diagnostics, spares/RMA, install-partner tooling.
- **K4. Site-survey and commissioning product.** Turn `CAMERA_PLACEMENT.md` and `CAPTURE_PROTOCOL.md` from documents a human follows into a guided workflow that *validates* compliance at install: feet-in-frame verification, mirror-in-frame detection, camera-spacing check, calibration capture and acceptance. The difference between a 10-site business and a 100-site one.
- **K5. SRE.** SLOs, on-call rotation, runbooks, incident review, status page, capacity planning.
- **K6. Per-site model/threshold adaptation.** New sites need tuning (lighting, ceiling heights, equipment, clientele) as a supported operation with a validation harness, not a code fork.

Exit criteria: 1. A new gym goes from contract to live using documented workflows, with a target commissioning time and no bespoke engineering. 2. Cross-tenant isolation verified by automated test and external penetration test. 3. Fleet update rolled out and rolled back across multiple sites without a member-visible incident. 4. On-call and SLOs operating with a reviewed incident history. 5. Billing reconciles cleanly, including a downtime-credit case.

Phase L — Model and coverage expansion (continuous, parallel)

Not a sequential phase — a track that runs alongside F–K and feeds each. Sourced directly from the existing `ROADMAP.md` backlog and `STATUS.md` limitations.

- **L1. Hip-driven rep counter.** Already in the `ROADMAP.md` backlog. Unlocks sit-ups (currently `CLASS_FAMILY['situp'] = 'none'`) and improves hinge movements. Directly expands what the product can promise.
- **L2. Expand trained classifier coverage.** Currently 7 trained classes; `hinge`, `lunge`, `curl`, `row` are heuristic-only. Domain shift is a live concern — `FIT3D_UTILIZATION_PLAN.md` Track B records the trained classifier badly misclassifying Fit3D footage in a small sample, and correctly notes the fix **cannot** be Fit3D fine-tuning (academic license). Real gym footage from the pilot is the licensed path, subject to Phase G8.
- **L3. Reference signatures for absolute form scoring.** Prerequisite for J3's honest "accuracy score." Per-exercise, built from good-form footage, same-camera or 3D.
- **L4. Mirror dead-zone masking.** `ROADMAP.md` A5 explicitly backlogs a per-camera polygon mask as "a machine-enforced backstop for whenever the placement rule isn't followed perfectly." **After Phase H there is an installation** — given commercial gyms are mirror-walled, this likely graduates from backlog to requirement.
- **L5. Detector upgrade under real occlusion.** yolov8n@640 detects both people only 30% of the time on heavily-occluded two-person footage vs 61% for yolov8s@960. Worth revisiting once H3's real hardware numbers are in hand. Also resolve the AGPL question from 0.10(b).
- **L6. 2-camera fused stations.** `ROADMAP.md` A3's accepted residual limit (1-DOF epipolar ambiguity at exactly 2 views) means 2-camera stations carry a real blind spot. `CAMERA_PLACEMENT.md` rule 4 already prefers 3+. Worth quantifying so sales knows what a 2-camera station can and cannot claim.

Part 3 — Architecture recommendations

3.1 Target shape

Three tiers, with a deliberate boundary between them:

```
EDGE (one per gym site)
  Floor Runtime Service (evolved pipeline.py)
    ingest workers   -> RTSP + timestamped frames
    inference pool   -> detector + pose (shared, batched)
    analysis actors  -> per-person state (state_machine, form,
                        risk, classifier, fusion)
    visit gallery    -> src/session.py, visit-scoped
    emitter          -> durable local spool -> upload

  RAW FRAMES NEVER LEAVE THIS BOUNDARY
        |  events + skeletons only
        v
CONTROL PLANE (cloud, multi-tenant)
  Ingest API (device auth, idempotent)
  Core API   (user auth, tenant-scoped)
  Relational store: members, visits, sets, reps, plans, form
  Bulk store: angle series, skeletons
  Plan/adherence engine, notifications, billing, catalog
        |
        v
  Member app  |  Operator console  |  Content CDN
```

3.2 Decomposing `CameraProcessor.run()`

Today that one method owns capture, detection, Re-ID resolution, pose, smoothing, angles, two state machines, classification, form scoring, visualization, dashboard push, and JSON writing — as one 250-line loop with all state in local dicts. A perfectly reasonable research script and a poor service.

Split along the seams the code already implies:

1. **Ingest** — one worker per camera; owns `cv2.VideoCapture`, reconnection, and stamping each frame with a real time (H2). Emits `(camera_id, wall_time, frame)`.
2. **Inference** — a shared pool owning models rather than one `PersonDetectorTracker` + `PoseEstimator` per camera thread. This is where batching (H4) becomes possible; today's per-camera model instances make it impossible by construction. Constraint: `PersonDetectorTracker` explicitly holds per-camera tracker state (`persist=True`), so **tracker state must stay per-camera even if model weights are shared** — `HANDOVER6.md` records a bug caused by exactly this (a shared ByteTrack instance across 4 feeds corrupting tracker state).
3. **Analysis** — per-person stateful actors. The existing `_init_person()` dict is already effectively an actor's state; make it one, visit-scoped rather than process-scoped.
4. **Emission** — events out (F3), not a file at the end.

Keep `src/state_machine.py`, `src/angles.py`, `src/form.py`, `src/risk.py`, `src/signature.py`, `src/triangulation.py` **untouched**. They are pure, well-tested, and carry the project's validated behaviour. The refactor should be entirely in the orchestration layer, with the Panoptic/Fit3D/ RepCount validations re-run as the regression gate.

3.3 Edge vs cloud inference: run inference at the edge

Recommendation: **edge**, with cloud reserved for aggregation, API, and model distribution. Three grounded reasons, in order of weight:

1. **Compliance.** If raw frames never leave the site, the cloud never holds video of people's bodies, and the regulatory surface shrinks enormously — DPIA scope, breach exposure, cross-border transfer, BIPA/GDPR analysis all get materially simpler. It preserves the honest version of the property `README.md` currently claims. Retrofitting this later is impossible.
2. **Bandwidth.** `DEVELOPMENT.md` estimates ~20 cameras for full coverage of a university gym, ~10 for a 2D-only MVP. Ten to twenty 1080p streams is a sustained multi-tens-of-Mbps upstream commitment, 12+ hours a day, forever. Skeleton and event data is orders of magnitude smaller.
3. **Failure mode.** A gym with a flaky uplink still tracks workouts and syncs later. A cloud-inference design goes dark.

The cost: H3's unmeasured hardware envelope becomes the binding constraint on the whole business model. If real Jetson throughput at real occupancy means one device per 2 cameras rather than per 4, per-site hardware cost roughly doubles. **This is why H3 is an exit-criteria- bearing measurement, not a footnote.**

Narrow cloud-inference exceptions worth keeping open: a short-retention, opt-in clip escrow for disputed sessions; offline reprocessing of consented footage for model improvement (Phase L, gated by G8).

3.4 Identity across visits: bind at check-in, never persist embeddings

The single most consequential architectural call in the plan.

Do not persist `SessionGallery` embeddings between visits. Technically, the embedding is ImageNet features plus a torso HSV histogram, explicitly designed around "clothing stays constant" within a session. Legally, a stored appearance template derived from a person's body is the archetype of a regulated biometric identifier; the current session-only design is a genuinely strong compliance position that should be preserved deliberately, not lost by accident.

Instead, three layers:

Layer	Scope	Mechanism	Persisted?
Tracklet	seconds–minutes	ByteTrack (`src/detector.py`)	No
Visit person	one visit	`SessionGallery` + `ProvisionalIdentity` (unchanged)	No (RAM only)
Member	forever	authenticated check-in → `bind(visit_person -> member_id)`	Yes (ID + binding record; no body data)

Consequences to design for: - `REID_STALE_SECONDS = 1800` means a member gone 40 minutes re-enrolls as a new `visit_person` — fine under this model, it just needs a re-binding path (app re-check-in, station-local tap). - The binding event needs recorded evidence and confidence, and a correction path (I6). Mis-binding must be a monitored, alertable rate. - Unbound `visit_person` records need a defined disposition. - Every rep in the database should carry its binding provenance, so a disputed session can be adjudicated rather than argued.

3.5 Data model shape

- **Relational (hot):** tenant, gym, site, camera, station, calibration, member, membership, consent, visit, visit_person, binding, session, set, rep, form_score, risk_assessment, plan, plan_item, scheduled_session, adherence_record. Small rows, high query volume, tenant column on every table.
- **Bulk (cold):** angle series, 3D skeletons, valgus series. Written once, read rarely, TTL'd aggressively. Keyed by session; never joined per-frame in the relational store.
- **Event log:** the append-only stream from F3, source of truth for reconstruction and late-arriving edge data.

Two schema rules learned directly from this repo: **(a)** carry absolute wall-clock time *and* the source-relative clock on every record — the relative clock is what the validated algorithms use, the absolute one is what the product needs; **(b)** do not model angle series as joint-aligned frames until F6 lands, or the per-joint drift in `src/output.py` will silently corrupt every downstream aggregate.

3.6 Configuration

`src/config.py` should become the *default layer* of a resolved configuration (defaults → tenant → site → camera), loaded at runtime. Keep current values as defaults verbatim so `tools/benchmark_repcount.py` , `tools/validate_panoptic.py` , and `tools/validate_fit3d.py` remain reproducible against published figures. Every resolved config should be versioned and attached to the sessions it produced.

3.7 Model serving

Keep models in-process at the edge. The real work is H3/H4: delegate/ TensorRT, batching, and the linear-in-people pose cost. Add model versioning with per-site pinning and rollback, and a validation harness that runs the existing Panoptic/Fit3D/RepCount benchmarks against any candidate model before it can be pinned anywhere — the project already has unusually good benchmark infrastructure in `tools/`; turning it into an automated model-acceptance gate is cheap and high-value.

Part 4 — Key technical risks

Product-specific risks — physical space, real bodies, paying members — not generic software risk.

4.1 Accuracy risks that constrain what an MVP may promise

R1 — Rep-count accuracy is not yet a product number. Current authoritative benchmark (`ROADMAP.md` B1): RepCount Part-A, all 152 videos, **MAE 0.742, OBO 22.4%**; gym-core classes squat 44.4%, front_raise 33.3%, pull_up 36.8%, push_up 31.2% OBO — uncontrolled internet footage, not `CAMERA_PLACEMENT.md`-placed cameras. On the project's own purpose-shot footage results were far better. Both facts are true, neither is a product number. **Only Phase I5's human-ground-truth audit produces the number marketing is allowed to use.**

R2 — Bench press cannot be reliably counted head-on. `STATUS.md`: counting is weak, "needs a side camera angle, not a code fix"; `CAMERA_PLACEMENT.md` rule 2 adds that 45° is insufficient — needs a true side-on view. One of the most-performed exercises in a commercial gym.

R3 — Sit-ups and hip-driven movements count zero, by design. `CLASS_FAMILY['situp'] = 'none'`. Correct engineering choice, and a product hole.

R4 — Mirrors are a total blind spot, and commercial gyms are mirror-walled. `ROADMAP.md` A5: nothing in the pipeline can distinguish a reflection from a person; two failure paths (contamination of a real person's state, or a phantom identity double-counting reps). `CAMERA_PLACEMENT.md` rule 3 makes "no mirrors in any camera's frame" a hard rule with no partial credit. **This may be physically unsatisfiable in a typical commercial free-weight area** — the highest-probability physical-deployment surprise in this plan.

R5 — Feet-in-frame is required and conflicts with crowded floors. Degradation is silent, not errored — plausible wrong numbers rather than a flag.

R6 — Crowding and occlusion. yolov8n@640 detected both people only 30% of the time on heavily-occluded two-person footage. A commercial floor is many-person occlusion continuously.

R7 — Domain shift, three ways. RepCount YouTube footage vs. Fit3D studio footage (already recorded as a bad misclassification pattern in a small sample) vs. a real gym vs. each site's own lighting/geometry. The obvious fix (fine-tune on Fit3D) is license-blocked.

R8 — Form scores are self-referential today. A "form score of 82" largely means "consistent with your own recent motion," not "82% correct." The product vision promises right/wrong plus an accuracy score.

R9 — 2-camera fused stations carry an unclosable blind spot. `ROADMAP.md` A3's accepted residual: at exactly 2 views a wrong cross-view match can measure 0.6px — indistinguishable from noise. Not fixable by tuning.

4.2 Identity, trust, and attribution risks

R10 — Mis-attribution is now a trust failure, not just a data error. `src/session.py` already names it "the worst failure mode." As a product, wrong attribution means a member sees someone else's workout, or their PR is credited to a stranger. With billing attached, it becomes a dispute.

R11 — Conservative matching produces fragmentation, which looks like data loss. A2's design deliberately enrolls a new anonymous person rather than guess; in a crowded gym with similarly-dressed members, a visit may fragment into several unbound records.

R12 — Adherence tracking risks accusing paying members. False negatives are structurally guaranteed by R2–R6.

R13 — Bystanders who never consented. Guests, staff, cleaners, contractors, minors — currently silently enrolled as `person_N`.

4.3 Platform, operational, and legal risks

R14 — The unauthenticated HTTP surface. `src/dashboard.py` binds `0.0.0.0` and serves live gym-floor JPEGs, a full identity roster, and a state-mutating `GET /checkin` with no auth, TLS, or audit. On a gym's flat network, that is a live video feed of people exercising available to anyone on the LAN.

R15 — Biometric-regulation exposure. BIPA's private right of action is the highest single-cost risk; GDPR Art. 9 plus a mandatory Art. 35 DPIA applies in Europe/UK. The current system is in a strong position precisely because it persists nothing — Phase F is what creates the exposure.

R16 — Health/injury claim creep. `src/risk.py` emits `risk_level: high` with signals named `fatigue_form_decline` and `chronic_asymmetry`. Marketing pressure to call this "injury prevention" will be constant, and crossing that line changes the regulatory and liability picture.

R17 — Silent calibration drift. A bumped camera invalidates the rig with no automatic detection today. A gym floor bumps cameras.

R18 — Data loss from the current durability model. No DB, in-memory unbounded state, one write at process exit. A crash at minute 80 of a 90-minute session loses the whole session — for a member, that's their workout gone.

R19 — Unknown unit economics until H3 measures throughput. 0.22 s/frame for 4 views with one person is the only measurement in the repo; pose is linear in people; Jetson is entirely unmeasured. This number determines hardware cost per site, which determines whether the business model works.

R20 — Third-party licensing. Ultralytics YOLOv8 (AGPL-3.0 with a commercial option) is the shipped detector; training-data licenses need confirming for the shipped classifier.

R21 — No test suite, no CI, no packaging. Everything in Phases F–K is a refactor of validated behaviour, and there is currently no automated way to prove the refactor didn't break it.

4.4 What an MVP can honestly promise

Can promise (subject to I5 confirming at the site): - Automatic counting of squats and knee-driven lower-body work, and elbow-driven upper-body work, at properly placed, mirror-free, feet-in-frame stations. - Per-set consistency and control feedback (tempo, symmetry, ROM consistency, trunk lean) with actionable coaching cues. - Session history across visits, with per-set confidence and coverage shown. - True 3D joint

angles and knee valgus at 3+ camera fused stations. - A plan builder, and adherence *for observed activity*, with self-logging for everything else. - A demo-video library for the full exercise catalog.

Must not promise at MVP: - Reliable bench-press counting without a true side camera (R2). - Any sit-up or core-flexion counting (R3). - Coverage of mirror-adjacent areas (R4). - An absolute "correctness" percentage, until L3/J3 lands (R8). - Injury prediction or any medical claim (R16). - Adherence assertions about unobserved activity (R12). - Any headline accuracy figure before I5 produces one (R1).

Appendix — Files referenced

Docs read: `ROADMAP.md`, `STATUS.md`, `DEVELOPMENT.md`, `README.md`, `CAMERA_PLACEMENT.md`, `FIT3D_UTILIZATION_PLAN.md`, `HANDOVER6.md`.

Code read: `pipeline.py`, `requirements.txt`, and in `src/`: `camera.py`, `fusion.py`, `session.py`, `reid.py`, `output.py`, `dashboard.py`, `config.py`, `state_machine.py`, `form.py`, `risk.py`, `pose.py`, `detector.py`, `exercise_classifier.py`.

Confirmed absent: `tests/`, `.github/`, `Dockerfile`, `docs/`, `models/references/` (only `models/exercise_clf.npz` exists).

Files with specific outstanding defects named in this plan: `src/output.py` (per-joint downsample clock drift, F6), `src/dashboard.py` (unauthenticated surface, G6), `src/config.py` (global constants blocking per-site tuning, H6).

No files were modified while researching this plan.

Joule Coach strategy document · prepared 6 August 2026 · compiled from PRODUCT_SPEC.md and PRODUCT_ROADMAP.md in the project repository.